

“SOCIETY/APARTMENT MANAGEMENT PORTAL”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

V.SUJIT

2520090085

M.SAI CHAITANYA

2520090023

D.BALA CHARAN

2520090087

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Society/Apartment Management Portal”** is a record of Bonafide work of **V. SUJIT (2520090085)** , **M. SAI CHAITANYA (2520090023)** , **D. BALA CHARAN (2520090087)** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 7, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

V.SUJIT– 2520090085

M.SAI CHAITANYA – 2520090023

D.BALA CHARAN -- 2520090087

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“Society/Apartment Management Portal”** is a Bonafide work done and submitted by **V. SUJIT (2520090085)** , **M. SAI CHAITANYA (2520090023)** , **D. BALA CHARAN (2520090087)** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 10, Team 7, at KLH University during **2026–2027, Trimester 4.**

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. Ramatenki Sateesh Kumar** , and my guide **Divya Priya Degala**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the *Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

V.SUJIT – 2520090085

M.SAI CHAITANYA – 2520090023

D.BALA CHARAN -- 2520090087

Abstract

The Society/Apartment Management Portal is a web-based application designed to simplify and digitize the day-to-day management activities of residential apartments and housing societies. In many apartments, activities such as maintenance fee collection, complaint handling, visitor entry, amenity booking, and communication with residents are managed manually or through separate systems. This can result in inefficient record management, delayed communication, and difficulty in tracking payments and requests.

The proposed system provides a centralized platform where apartment administrators and residents can efficiently manage these activities. The portal maintains resident and flat information and allows administrators to generate and track maintenance bills. Residents can view their maintenance dues and make online payments through the integrated payment facility. The system also supports visitor gate-pass management for maintaining visitor records and improving entry management.

The system will use a structured database to maintain resident, payment, complaint, visitor, and booking information. Overall, the project aims to provide an organized, convenient, and centralized solution for managing residential society operations.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	9
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	10
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	12
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	13
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	15
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	17
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	18

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	19
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	20
13	References	– Documentation, tools, and resources used	21
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	22

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Residential apartment societies involve several recurring management activities. These include maintaining resident and flat information, collecting maintenance charges, handling complaints, managing visitors, booking common amenities, and communicating important information to residents.

In a traditional apartment management system, these activities may be performed manually or through multiple independent systems. Such an approach can make it difficult for administrators to maintain accurate records and for residents to track their requests and payments.

The **Society/Apartment Management Portal** is proposed as a centralized web-based platform that brings these activities together into a single system.

The portal is designed to provide separate functionality for apartment administrators and residents. Administrators can manage resident information, flats, maintenance bills, complaints, visitors, and amenity-related activities, while residents can view their information, check maintenance dues, make payments, submit complaints, manage visitor requests, and access relevant apartment information.

1.2 Problem Statement

Managing residential apartments and housing societies often involves handling maintenance fee collection, complaint management, visitor entry, amenity booking, and communication with residents through manual records or separate systems. This can lead to inefficient record management, delayed communication, difficulty in tracking payments and requests, and challenges in maintaining accurate resident and visitor information. Therefore, there is a need for a centralized **Society/Apartment Management Portal** that can digitize and simplify these activities, allowing administrators and residents to efficiently manage apartment operations, payments, complaints, visitor records, and bookings through a single platform.

1.3 Scope of the Project

The scope of the Society/Apartment Management Portal includes management of residents and flats, maintenance bill generation and tracking, online maintenance payment support, complaint management, visitor gate-pass and visitor record management, amenity booking, and communication between administrators and residents. The system is intended to provide a centralized database for storing important society information and to make routine management activities more organized and accessible. The project focuses on the core requirements of residential apartment management and can provide a foundation for further expansion with additional society-management features in the future.

1.4 Objectives

The main objective of the Society/Apartment Management Portal is to develop a centralized web application that simplifies the management of residential society operations. The system aims to maintain resident and flat information in an organized manner, enable administrators to generate and track maintenance bills, allow residents to view their dues and make online payments, manage complaints and requests, maintain visitor records through gate-pass management, support amenity booking, and improve communication between administrators and residents. Another important objective is to use a structured database to store and retrieve society-related information efficiently.

1.5 Importance of the Application

The application is important because it provides a centralized and organized approach to residential society management. Instead of depending on manual records or multiple separate systems, administrators and residents can use one platform to manage important activities. The system can improve accessibility of resident and flat information, make maintenance dues easier to track, simplify payment management, maintain visitor records, organize complaints and bookings, and improve communication. By digitizing these activities, the portal aims to make society operations more convenient and easier to manage.

1.6 Target Users / Audience

The primary users of the Society/Apartment Management Portal are **apartment administrators and residents**. Administrators are responsible for managing resident and flat information, generating and tracking maintenance bills, monitoring payments, handling complaints, maintaining visitor information, and managing other society operations. Residents use the portal to view their information and maintenance dues, make online payments, submit or track complaints, manage visitor-related requests, and use available amenity-booking facilities. The system is therefore designed to support both administrative management and resident access within a residential society

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The Society/Apartment Management Portal is a web-based application and therefore requires a computer or laptop for development and administration, along with devices such as smartphones, tablets, or computers for residents to access the portal through a web browser. A development system should have sufficient processor performance, RAM, storage, keyboard, mouse, and network connectivity to run the frontend, FastAPI backend, database tools, development environment, and testing tools. Since the application uses a client-server architecture, users also require a stable internet or local network connection depending on the deployment environment.

2.2 Software Requirements

The proposed system uses **React.js or HTML/CSS/JavaScript** for the frontend, **FastAPI with Python** for the backend, and **MongoDB** as the database. The database can be accessed using **Motor, PyMongo, or Beanie**, depending on the implementation approach. Development and API testing can be performed using **VS Code, Git, Postman, and Swagger UI**, with Swagger UI being available through FastAPI for API documentation and testing.

2.3 Development Tools and Frameworks

The development of the Society/Apartment Management Portal uses a combination of frontend, backend, database, and development tools. React.js can be used to create an interactive user interface, while FastAPI provides the backend API layer for processing requests and communicating with MongoDB. MongoDB is used for storing resident, payment, complaint, visitor, and booking information, while Motor, PyMongo, or Beanie can be used as the database driver or ODM. Postman and Swagger UI support API testing, Git provides version control, and VS Code can be used as the primary development environment.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend of the Society/Apartment Management Portal provides the user interface through which administrators and residents interact with the system. React.js can be used to create reusable components and dynamic pages for functions such as resident management, maintenance bills, payments, complaints, visitor management, and amenity booking. Alternatively, simple HTML, CSS, and JavaScript can be used for a basic implementation. The frontend communicates with the FastAPI backend through APIs and displays the data received from the backend in a user-friendly manner.

3.2 Backend

The backend of the system is developed using **FastAPI with Python**. It acts as the main processing layer between the frontend and the database and provides APIs for different operations of the portal. The backend can handle requests related to residents, flats, maintenance bills, payments, complaints, visitors, and amenity bookings. FastAPI also provides built-in support for API documentation through Swagger UI, making it useful for developing and testing the backend services.

3.3 Database

MongoDB is used as the database for the Society/Apartment Management Portal. The database stores structured information required for society operations, including resident information, flat details, maintenance payments, complaints, visitor records, and amenity bookings. A suitable database structure allows the application to store and retrieve information efficiently while keeping the different types of society records organized. The application can communicate with MongoDB through Motor, PyMongo, or Beanie.

3.4 Version Control

Git can be used as the version-control system for the project. It allows the development team to maintain different versions of the source code and track changes made during development. Team members can use Git to manage frontend and backend code, maintain development history, and collaborate while building the Society/Apartment Management Portal. Version control also helps in restoring previous versions of the project when required.

3.5 Authentication and Security Libraries

Authentication and authorization are important for the portal because different users such as administrators and residents should have access to functions relevant to their roles. The backend can implement secure user authentication and role-based authorization so that administrative operations are protected from unauthorized access. User credentials and sensitive application data should be handled securely, while API endpoints should validate incoming requests before performing database operations. The exact authentication library is not specified in the submitted abstract, so the final implementation should document the specific library selected by the development

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The Society/Apartment Management Portal follows a layered web application architecture consisting primarily of the **frontend**, **backend API**, and **database layers**. The administrator or resident interacts with the frontend through a web browser, the frontend sends requests to the FastAPI backend, and the backend processes those requests and communicates with MongoDB for storing or retrieving information. The backend then sends the required response to the frontend, which presents the result to the user. This architecture separates the user interface, application logic, and data-storage responsibilities, making the system easier to organize and maintain.



Figure 4.1: High-level three-tier architecture of the Event and Room Occupancy Management System

4.2 Description of Each Layer

The **frontend layer** provides the interface for administrators and residents and contains pages for society-related operations such as maintenance, complaints, visitors, payments, and bookings. The **backend layer**, developed using FastAPI and Python, manages application logic, API requests, authentication, validation, and communication with the database. The **database layer**, based on MongoDB, stores resident, flat, payment, complaint, visitor, and booking information. Together, these layers provide a centralized system for managing residential society activities.

4.3 Deployment Architecture

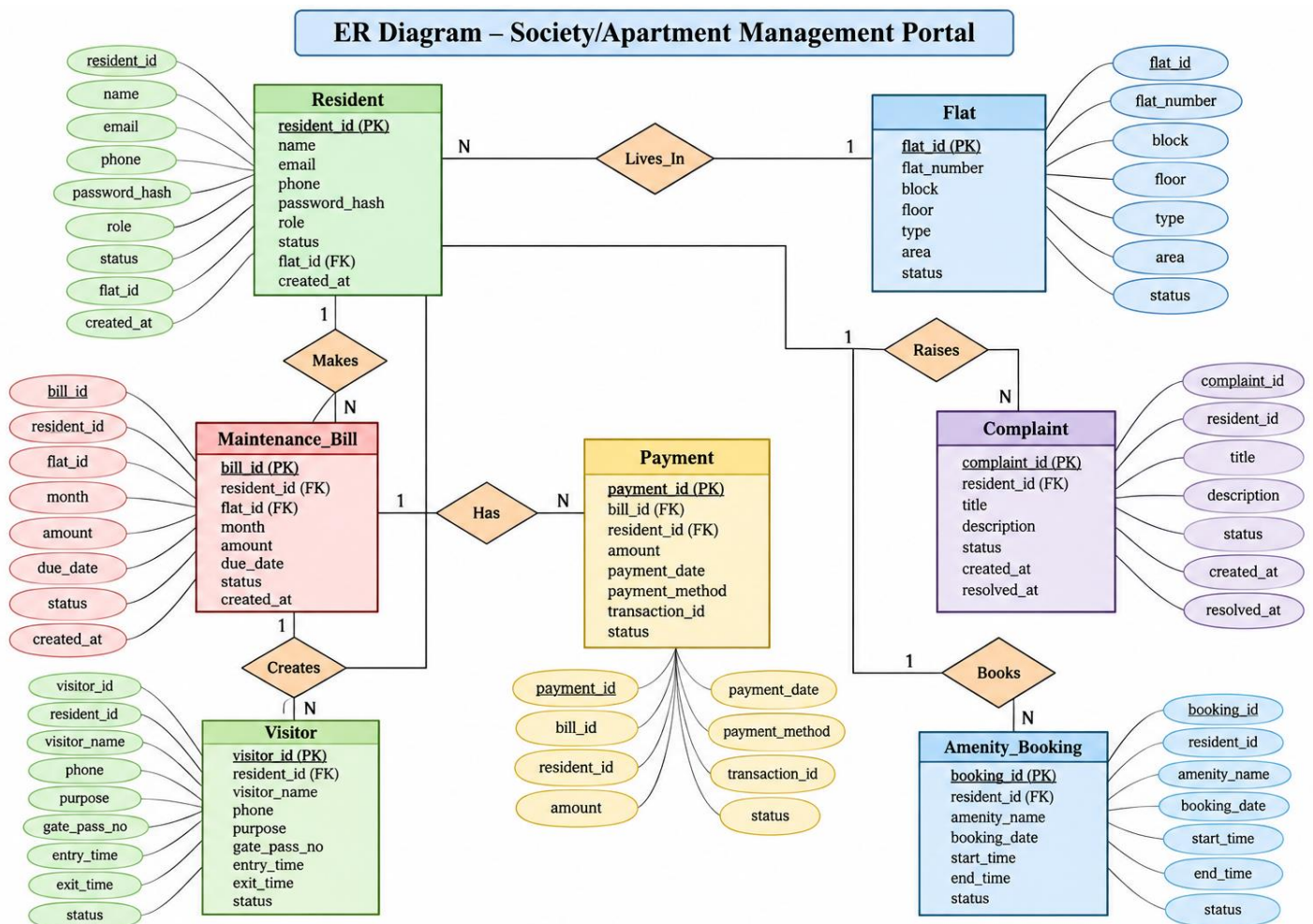
The deployment architecture consists of a client side through which users access the portal and server-side components responsible for processing application requests and managing data. The frontend can be hosted as a web application, while the FastAPI backend runs as an API service connected to the MongoDB database. Residents and administrators can access the application using supported web browsers through a network connection. The exact hosting platform is not specified in the project abstract, so the final deployment environment can be documented according to the platform selected during implementation.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The database design of the Society/Apartment Management Portal focuses on maintaining organized and accessible information about residents, flats, payments, complaints, visitors, and bookings. The database should avoid unnecessary duplication of information and should maintain appropriate relationships or references between relevant records. Data validation, consistent naming, unique identifiers, and appropriate indexing can be used to improve data integrity and retrieval performance. Since MongoDB is used, collections can be organized according to the major entities handled by the application.

5.2 ER Diagram / Database Schema



The database schema for the portal can contain major entities such as Resident, Flat, Maintenance Bill, Payment, Complaint, Visitor, and Amenity Booking. A resident can be associated with a particular flat, while maintenance bills and payments can be associated with the respective resident or flat. Complaints can contain information about the resident who submitted them and their current status, while visitor records can contain visitor and gate-pass information. Amenity bookings can associate residents with available facilities and booking schedules. These entities together represent the major information required for residential society management.

5.3 Design Decisions and Assumptions

The major design decision is to use a centralized web-based application with separate frontend, backend, and database layers. React.js or HTML/CSS/JavaScript is considered for the frontend, FastAPI with Python is used for backend development, and MongoDB is selected for data storage. The system assumes that apartment administrators and residents have authorized accounts and that the administrator is responsible for managing society-level information. The portal is designed around the activities identified in the project abstract, including maintenance management, payments, complaints, visitor records, amenity booking, and communication.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The Society/Apartment Management Portal can be implemented as a collection of modules corresponding to the major society-management activities. The Resident and Flat Management Module maintains resident and flat information, the Maintenance Management Module handles maintenance bill generation and tracking, the Payment Module allows residents to view dues and make online payments, the Complaint Management Module handles resident complaints, the Visitor Management Module maintains visitor and gate-pass records, and the Amenity Booking Module manages bookings for common facilities. A communication facility can also be provided for sharing information between administrators and residents.

6.2 Front-end Logic

The frontend provides different screens and user interfaces according to the role and requirements of the user. Residents can access information such as their maintenance dues, payment details, complaints, visitor requests, and amenity bookings, while administrators can access management functions related to residents, flats, bills, payments, complaints, visitors, and bookings. The frontend sends user actions to the FastAPI backend through API requests and displays the responses received from the server. React.js can be used to manage components, forms, navigation, and dynamic data presentation.

6.3 API Integration

API integration connects the frontend of the portal with the FastAPI backend. APIs can be created for operations such as adding or retrieving resident information, managing flats, generating maintenance bills, recording payments, submitting complaints, managing visitor information, and creating amenity bookings. When a user performs an operation on the frontend, the appropriate API request is sent to the backend, where the request is validated and processed before interacting with MongoDB. FastAPI's Swagger UI can be used to document and test these APIs.

6.4 Authentication and Authorization

Authentication and authorization provide controlled access to the Society/Apartment Management Portal. Users should log in using their authorized credentials, and the system should determine whether the user is an administrator or resident before providing access to protected operations. Administrators require access to management functions, whereas residents require access to their own relevant information and services. API-level authorization should be applied to prevent users from accessing functions or records that are outside their permitted role.

CHAPTER 7: FEATURES

7.1 List of Main Features

The main features of the Society/Apartment Management Portal include resident and flat information management, maintenance bill generation and tracking, maintenance dues viewing, online payment support, complaint handling, visitor gate-pass management, visitor record maintenance, amenity booking, and communication between administrators and residents. The system also provides centralized storage of resident, payment, complaint, visitor, and booking information. These features are intended to reduce manual management and provide a convenient platform for residential society operations.

7.2 How Each Feature Works

Room availability checking: given a date, a start time, an end time, and an optional minimum capacity, the system checks every room's status (excluding Maintenance/Inactive rooms), its capacity, and whether any existing room_schedules entry overlaps the requested window, returning two lists — rooms that are available, and rooms that are not, each with a plain-English reason such as “Already booked for ‘AI Workshop’ from 10:00 AM to 12:00 PM.”

CHAPTER 8: TESTING

8.1 Postman API Testing

A complete Postman collection was built covering all 22 REST endpoints, organized into six folders (Auth, Rooms, Events, Registrations, Bookings, Occupancy) plus a Health check. The Login requests carry a small test script that automatically saves the returned JWT into a collection variable, and requests such as Create Event, Assign Room, Register, and Create Occupancy Record are chained together using those variables, so a full workflow can be exercised without manually copying any ID or token between requests.

8.2 Integration Testing

Beyond Postman, the system was tested end-to-end against a real, running MySQL database at every stage of development — each backend module was verified with the actual database before the next module was built on top of it, rather than being tested in isolation with mocked data. As a final integration check, the entire database was dropped and rebuilt from `schema.sql` and `seed.sql` from zero, and the full checklist below was re-run against that fresh install to confirm the system works correctly for a first-time setup, not just in an already-running development environment.

8.3 Tools Used for Testing

- Postman — manual and scripted API testing
- curl (command line) — scripted regression testing across every phase of development
- MySQL command-line client — direct verification of table contents and constraint behavior
- Manual browser testing — verifying the React frontend's behavior and appearance

8.4 Test Cases and Results

A representative sample of the test cases executed, drawn from the project's actual test log, is summarized below.

#	Test Case	Expected Result	Actual Result
1	Login with wrong password	401, login rejected	Pass
2	Student token calls an admin-only route	403 Forbidden	Pass
3	Register for the same event twice	Second attempt returns 409 Conflict	Pass
4	Assign a Maintenance-status room to an event	409, room status rejected	Pass
5	Assign a room smaller than expected participants	409, capacity message with exact numbers	Pass
6	Assign a room already booked for an overlapping time	409, conflict message naming the clashing event	Pass
7	Request a room for a slot starting exactly when another ends	No conflict (back-to-back allowed)	Pass

#	Test Case	Expected Result	Actual Result
8	Create a room with negative capacity	400, rejected before reaching the database	Pass
9	Create a room with a duplicate room number + building	409, friendly duplicate message (no raw SQL error)	Pass
10	Reduce a room's capacity below an upcoming event's needs	409 warning listing the affected event(s)	Pass
11	Delete a room with existing booking history	Room marked Inactive instead of deleted	Pass
12	Delete an event with registrations and a room assignment	Cascades cleanly; zero orphaned rows afterward	Pass
13	Create an occupancy record with a negative headcount	400, rejected	Pass
14	Occupancy percentage at 45% / 95% / 108%	Labeled Normal / Near Capacity / Over Capacity	Pass

One test also uncovered a real defect rather than confirming correct behavior: pre-filling the Edit Event form's date/time fields initially used the API's raw UTC-encoded timestamp instead of converting it to the browser's local time, which would have shown an incorrect time (for example, 4:30 AM instead of 10:00 AM) on any machine not running in the UTC timezone. This is discussed further in Chapter 10.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is deployed locally for development and demonstration as follows:

- Load the schema and seed data: `mysql -u root -p < database/schema.sql`, then ... `< database/seed.sql`
- Start the backend: `cd server, npm install, npm run dev` (listens on port 5000)
- Start the frontend: `cd client, npm install, npm run dev` (listens on port 5173)
- Open `http://localhost:5173` in a browser and log in with a seeded account

9.2 Environment Configuration

All server configuration is kept in a single `.env` file (never committed to version control), covering the database connection (host, port, user, password, database name), the JWT signing secret and token expiry, the port the server listens on, and the frontend's URL for CORS. A `.env.example` template with placeholder values is committed instead, so the required variables are documented without exposing any real credentials.

9.3 Hosting of Frontend and Backend

For this submission, both the frontend and backend run as local Node.js processes and the database runs as a local MySQL instance, which is appropriate for development and in-class demonstration. Vite's development proxy forwards frontend API calls to the backend, avoiding any cross-origin configuration during development. A production deployment was not required for this course submission; Chapter 11 outlines how the same codebase could be adapted to a hosted environment.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Ambiguity between ROOM_SCHEDULES and BOOKINGS: the original data model included two tables that could both plausibly track “room + time,” which risked the two falling out of sync if both were updated independently.
- A timezone-conversion bug: the Edit Event form's date/time fields were pre-filled by slicing digits directly out of the API's UTC-encoded timestamp string, rather than converting it to the browser's local time first — correct only by coincidence when the server happens to run in the UTC timezone.
- A MySQL startup failure (“unable to setup unix socket lock file”) encountered mid-development, caused by a stale mysqld process still holding the socket file after an earlier restart.

10.2 Solutions Applied

For the ROOM_SCHEDULES/BOOKINGS ambiguity, a clear rule was adopted and enforced consistently in code: ROOM_SCHEDULES is the single source every conflict check reads from, and BOOKINGS is written at the same time, inside the same database transaction, purely as an audit trail — so the two can never disagree.

For the timezone bug, a small shared utility (dateUtils.js) was written that converts an ISO timestamp to a form field's expected value using JavaScript's local-time Date getters (getHours, getDate, and so on) rather than UTC getters or raw string slicing. The fix was verified by deliberately simulating a non-UTC server timezone (Asia/Kolkata) and confirming the displayed time matched what was actually stored.

For the MySQL startup failure, the stale process was terminated, the leftover socket and PID files were removed, and the service was restarted cleanly — now documented as a known troubleshooting step in the project's README.

10.3 Current Limitations or Known Bugs

- No email verification or password-reset flow currently exists for user accounts.
- Admin accounts can only be created by inserting a row directly into the database — by design, there is no public sign-up path to an admin role.
- Actual occupancy must be entered manually by an admin after an event; there is no automatic check-in mechanism (e.g. QR-code scanning).
- The system has been tested on a single local deployment and has not yet been deployed to a public cloud environment.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Email-based password reset and account verification.
- A safe, audited admin UI for promoting an existing user to an admin role, rather than requiring direct database access.
- A calendar view of room schedules, so an admin can see a room's bookings visually across a week or month.
- Email or SMS notifications for registration confirmation and event reminders.
- QR-code based check-in at events, so actual occupancy is captured automatically instead of typed in by hand.

11.2 Possible Integrations or Optimizations

- Integrating a calendar library (such as FullCalendar) on the frontend for the room-schedule view above.
- Caching frequently-requested room-availability queries to reduce repeated database load during peak registration periods.
- Containerizing the application with Docker for a consistent, reproducible deployment environment.
- Adding an automated test suite (e.g. Jest and Supertest) to complement the manual Postman and curl-based testing already performed, so regressions are caught automatically in future changes.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to replace informal, error-prone room coordination on a college campus with a single, database-driven web application. Over the course of development, a normalized seven-table MySQL schema, a 22-endpoint Express REST API, and a full React frontend for both students and administrators were designed, built, and tested incrementally, with each layer verified against a real, running database before the next was added.

12.2 What Was Achieved

The finished system enforces role-based authentication; prevents a room from ever being double-booked through one consistently-applied overlap rule; enforces registration rules (no duplicates, capacity-aware limits); gives administrators a live dashboard and full management of rooms, events, bookings, registrations, and occupancy; and was verified with a genuine, documented testing process — including a full fresh-database rebuild and a systematic pass through a final quality checklist — rather than being assumed correct.

12.3 Skills Learned

Working on this project involved practical experience in relational database design and normalization; writing and enforcing constraints at the database layer rather than trusting application code alone; designing a REST API with consistent response conventions and meaningful HTTP status codes; managing multi-table database transactions to keep related data consistent; building a React frontend with Context-based state management; and, importantly, systematic testing — including deliberately trying to break the system's own rules to confirm they hold, which is how the timezone bug described in Chapter 10 was found before it reached a real user.

CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev>
- Vite — Official Documentation, <https://vitejs.dev>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- Express.js — Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- mysql2 (Node.js MySQL driver) — npm documentation, <https://www.npmjs.com/package/mysql2>
- jsonwebtoken — npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- bcrypt — npm documentation, <https://www.npmjs.com/package/bcrypt>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs — JavaScript and Web APIs reference, <https://developer.mozilla.org>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

The core room-conflict overlap query, used consistently across the system:

```
// The single overlap rule used everywhere a room's schedule is checked
// (room availability search, event room-assignment, booking creation):
SELECT rs.room_id, rs.start_time, rs.end_time, e.event_name
FROM room_schedules rs
JOIN events e ON rs.event_id = e.event_id
WHERE rs.status = 'Scheduled'
      AND rs.start_time < ?    -- requested end time
      AND rs.end_time   > ?;  -- requested start time
```

Appendix B: Installation and Setup Instructions

- `mysql -u root -p < database/schema.sql`
- `mysql -u root -p < database/seed.sql`
- `cd server && npm install && npm run dev` (backend, port 5000)
- `cd client && npm install && npm run dev` (frontend, port 5173)
- Open `http://localhost:5173` in a browser

Appendix C: Default Login Credentials

All seeded accounts use the password “password123”.

Role	Email
Admin	admin@college.edu

Appendix D: API Reference Summary

22 endpoints across six resources; full request/response examples are in the accompanying Postman collection.

Resource	Endpoints
Auth	POST /register, POST /login, GET /me
Rooms	GET /, GET /:id, GET /available, POST /, PUT /:id, DELETE /:id
Events	GET /, GET /:id, POST /, PUT /:id, DELETE /:id, PUT /:id/assign-room
Registrations	POST /, GET /my, GET /event/:eventId, GET /
Bookings	GET /, POST /, PUT /:id, DELETE /:id
Occupancy	GET /, POST /, PUT /:id

Appendix E: Screenshots

The following pages are left for the student to complete with actual screenshots taken from the running application before submission. Suggested screenshots, in order:

1. Login Page

The image displays two screenshots of the Courtyard Society Living System interface. The left screenshot shows the home page, and the right screenshot shows the authentication portal.

Home Page (Left Screenshot):

- Header:** COURTYARD SOCIETY LIVING SYSTEM, MAPLE HEIGHTS - 2026
- Section:** SOCIETY OPERATIONS, MADE LIGHTER
- Headline:** Home has a rhythm.
- Text:** One clear place for the dues, maintenance requests, visitor passes, and decisions that keep your building moving smoothly.
- Buttons:** Direct Payments, Helpdesk & Requests, Visitor Gate-Pass
- Footer:** Encrypted Society Data & Role-Isolated Access, v2.4.0 - Production

Authentication Portal (Right Screenshot):

- Section:** AUTHENTICATION PORTAL
- Headline:** Sign in to your portal
- Text:** Select your doorway to access tailored building features.
- Buttons:** RESIDENT, ADMIN
- Form Fields:** FLAT / APARTMENT UNIT (Tower B - 704), REGISTERED MOBILE NUMBER (+91 98765 43210), PASSCODE / PIN (*****), Forgot PIN?
- Button:** ENTER RESIDENT PORTAL →
- Section:** INSTANT DEMO ACCESS
- Buttons:** Resident Demo (Bldg: Tower B-704), Admin Demo (Maple Heights Admin)

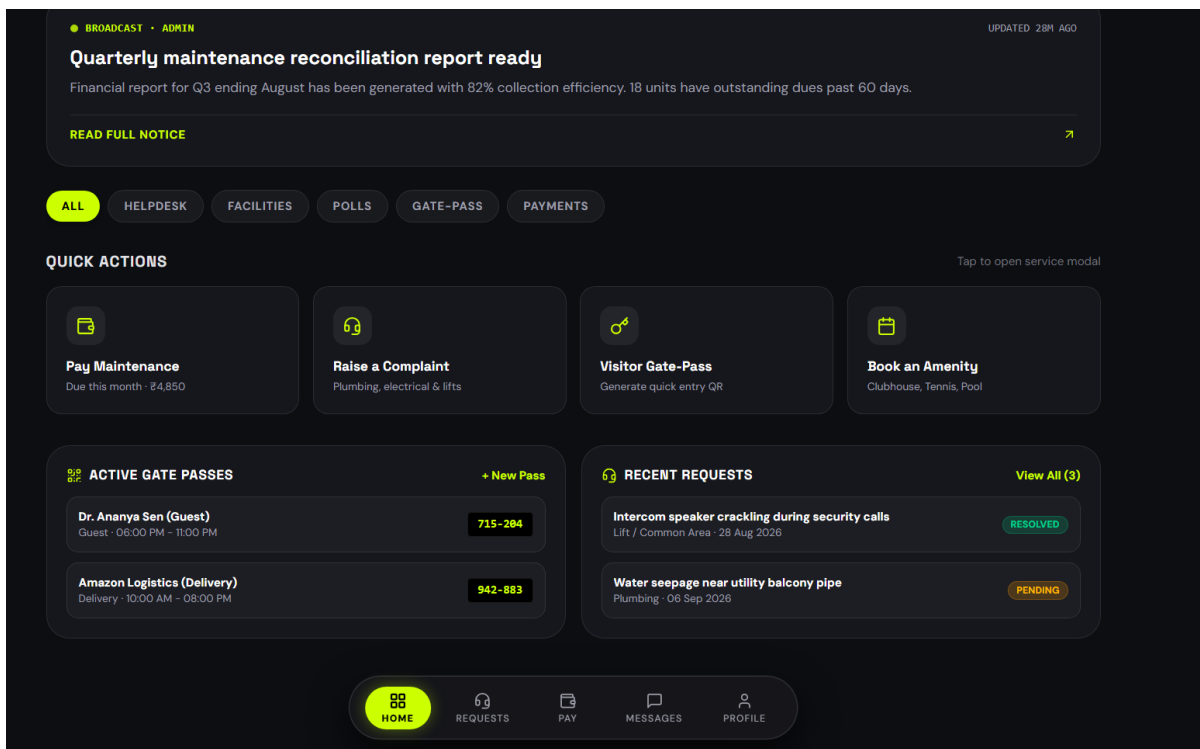
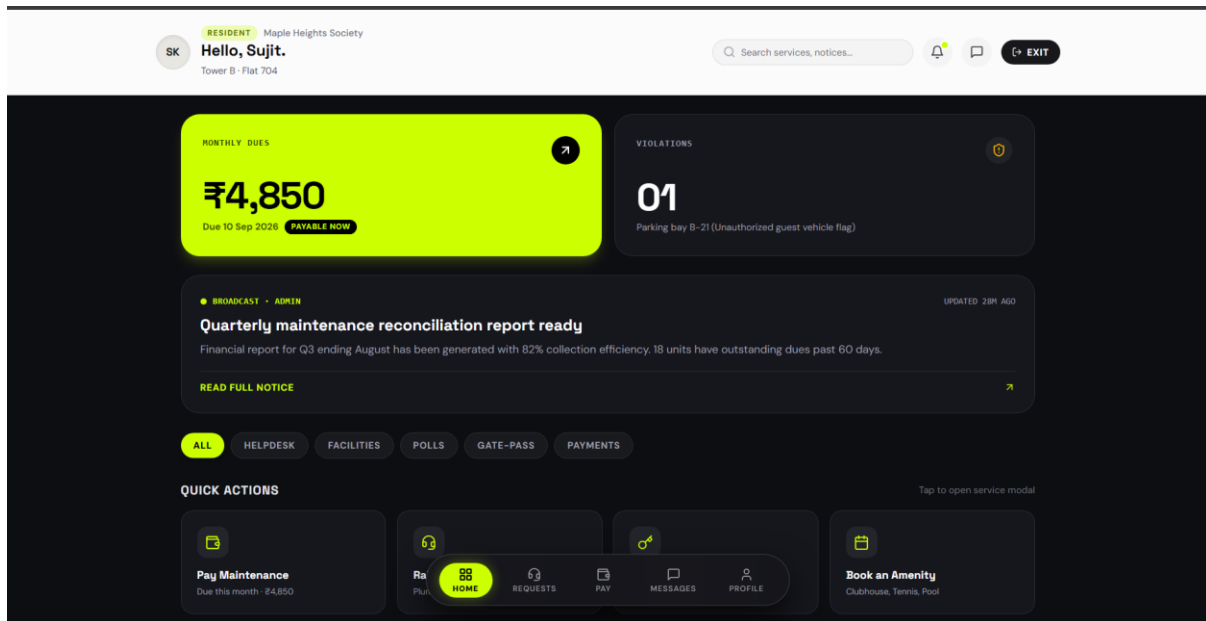
Home Page (Left Screenshot):

- Header:** COURTYARD SOCIETY LIVING SYSTEM, MAPLE HEIGHTS - 2026
- Section:** SOCIETY OPERATIONS, MADE LIGHTER
- Headline:** Home has a rhythm.
- Text:** One clear place for the dues, maintenance requests, visitor passes, and decisions that keep your building moving smoothly.
- Buttons:** Direct Payments, Helpdesk & Requests, Visitor Gate-Pass
- Footer:** Encrypted Society Data & Role-Isolated Access, v2.4.0 - Production

Authentication Portal (Right Screenshot):

- Section:** AUTHENTICATION PORTAL
- Headline:** Sign in to your portal
- Text:** Select your doorway to access tailored building features.
- Buttons:** RESIDENT, ADMIN
- Form Fields:** ADMINISTRATOR EMAIL (admin@mapleheights.org), MASTER SECURITY KEY (*****), HARDWARE 3FA ACTIVE
- Button:** ENTER ADMIN CONSOLE →
- Section:** INSTANT DEMO ACCESS
- Buttons:** Resident Demo (Bldg: Tower B-704), Admin Demo (Maple Heights Admin)

2. Dashboard



RESIDENT

Maple Heights Society

SK

Hello, Sujit.

Tower B · Flat 704

Q Search services, notices...

EXIT

Resident Profile & Unit Details

Account records registered with Maple Heights Estate Office

SK

Sujit

Tower B · Flat 704

OWNER RESIDENT · MAPLE HEIGHTS

Registered Phone

+91 98765 43218

Registered Email

sujit.k@courtyard.live

Allocated Parking Slot

Bay B-21 (Basement 1)

Registered Vehicle

MH-02-CD-8842 (Sedan)

Intercom Extension

Ext. 704

Family Members

Registered co-occupants (2)

+ ADD MEMBER

Aarav Vakkalanka

SON

Age: 4 yrs

Edit Remove

Ananya Vakkalanka

SPOUSE

Age: 28 yrs

+91 98765 43211

ananya.v@courtyard.live

EMERGENCY CONTACT

Edit Remove

LOG OUT

HOME

REQUESTS

PAY

MESSAGES

PROFILE

DUES CLEARANCE

Pay Society Maintenance

Total Dues for September 2026

₹4,850

Base Maintenance

₹3,500

Sinking Fund Contribution

₹600

Water Utility Charges

₹450

Reserved Parking Bay B-21

₹300

SELECT PAYMENT OPTION

Instant UPI

Card

NetBanking

UPI ID / VPA

sujit@okhdfcbank

256-Bit SSL Encrypted Society Escrow Account

PAY ₹4,850 NOW

Page 24

HELPODESK REQUEST

Raise a Complaint / Ticket

ISSUE CATEGORY

Plumbing

Electrical

Parking

Lift / Common Area

Security

Other

BRIEF SUMMARY

e.g. Geyser switch sparking in master bath

URGENCY LEVEL

Medium (Within 24h)

UNIT LOCATION

Tower B - Flat 704

DETAILED DESCRIPTION

Please describe the issue in detail...

SUBMIT TICKET TO FACILITY DESK

SECURITY GATE ACCESS

Create Visitor Gate-Pass

VISITOR TYPE

Guest

Delivery

Cab

Service Provider

VISITOR / DELIVERY AGENT NAME

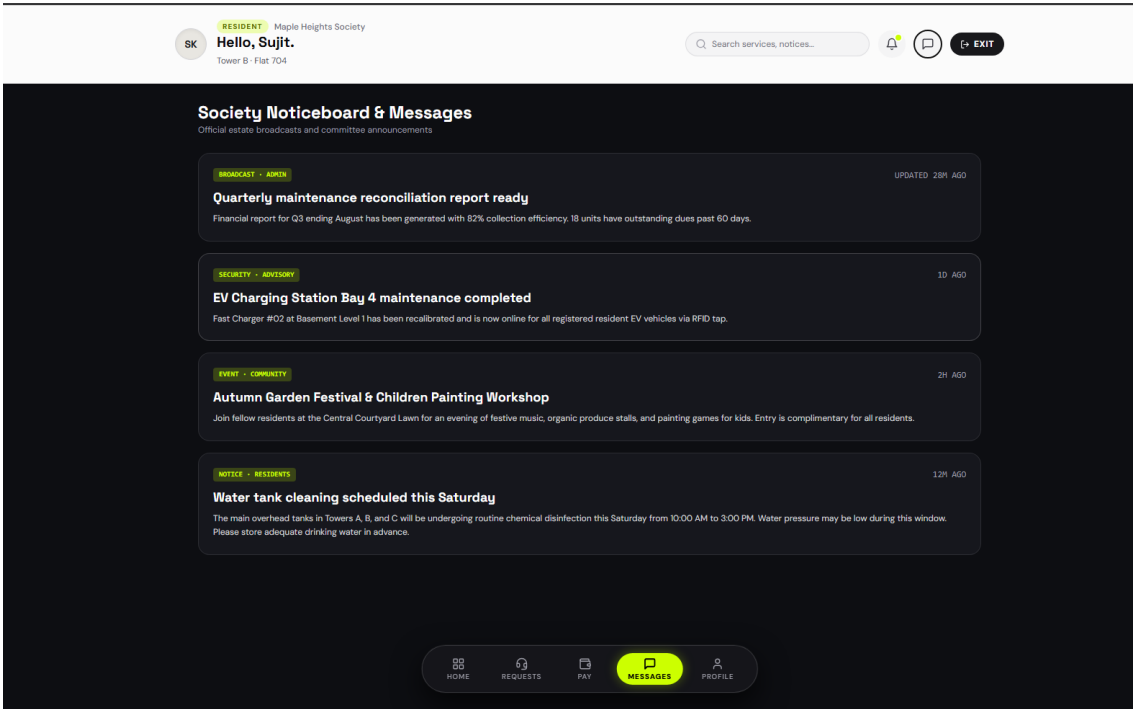
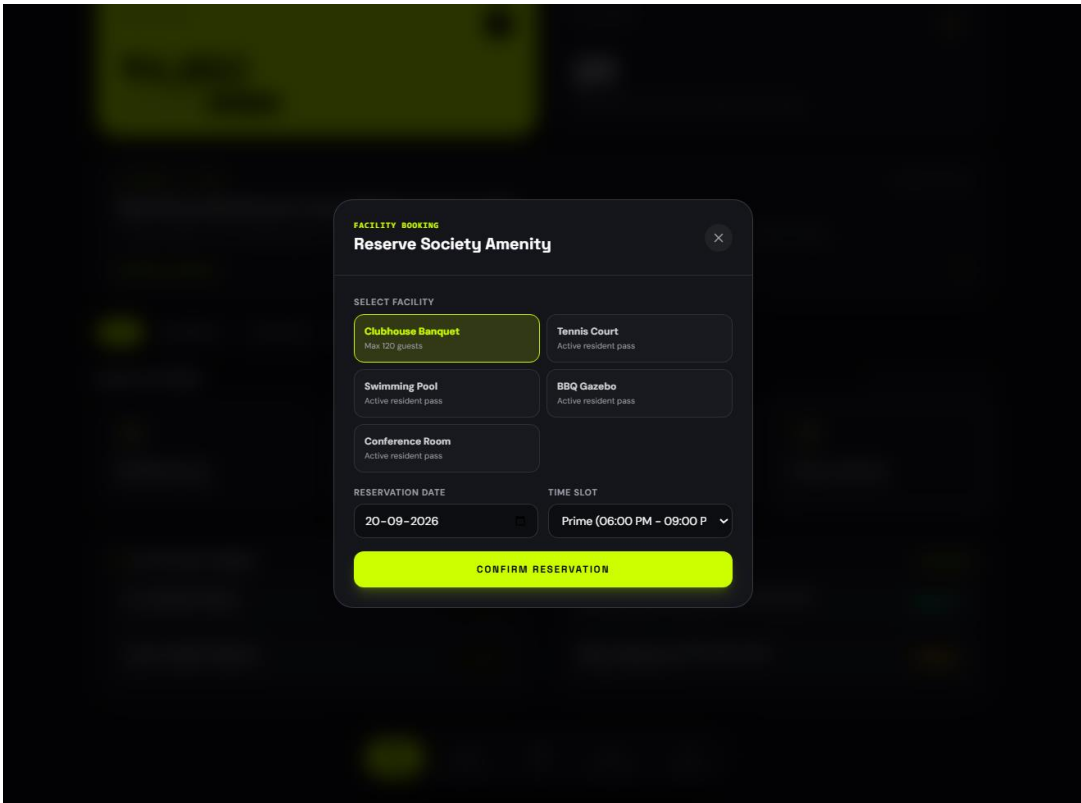
e.g. Zomato / Ramesh Sharma

VISITOR CONTACT PHONE (OPTIONAL)

+91 98765 00000

Security gate guards will scan the QR code for touchless barrier lift.

GENERATE INSTANT PASS QR



ADM

ADMIN CONSOLE

Maple Heights Society

Society Operations Desk

Admin Console - 248 Homes

BROADCAST NOTICE

EXIT

Resident & Family Member Directory

Manage resident records, tower allocations, capacity limits, vacant flats, and family members

MANAGE TOWERS

ADD RESIDENT

TOWERS & WING VACANCY OVERVIEW (3 TOWERS)

ADD TOWER

Tower A

84 Units

14 Floors - Residential Tower A (14 Floors, 6 units/floor)

2 Occupied

82 Vacant

2% Occupancy

82 Flats Available

Tower B

84 Units

14 Floors - Residential Tower B (14 Floors, 6 units/floor)

2 Occupied

82 Vacant

2% Occupancy

82 Flats Available

Tower C

80 Units

16 Floors - Residential Tower C (16 Floors, 5 units/floor)

2 Occupied

78 Vacant

3% Occupancy

78 Flats Available

Search residents by name, unit, phone, or email...

SEARCH

TOWER

All

Tower A

Tower B

Tower C

STATUS

All

Active

Inactive

SHOWING 6 RESIDENT RECORDS

RESIDENT & UNIT	CONTACT	TYPE	FAMILY MEMBERS	STATUS	ACTIONS
DR Dr. Vikram Mehra Tower A - Flat 101	+91 98281 11223 vikram.m@mapleheights.org	Owner Resident	2 Members	ACTIVE	
PO Pooja Agarwal Tower A - Flat 204	+91 98282 22334 pooja.a@mapleheights.org	Owner Resident	1 Members	ACTIVE	
SU Sujit Tower B - Flat 704	+91 98765 43210 sujit.s@mapleheights.org	Owner Resident	2 Members	ACTIVE	
AM Amitabh Sen Tower B - Flat 705	+91 98283 33445 amitabh.s@mapleheights.org	Tenant Resident	0 Members	ACTIVE	
KA Kavita Nair Tower C - Flat 302	+91 98284 44556 kavita.n@mapleheights.org	Owner Resident	0 Members	ACTIVE	
RO Rohit Deshmukh Tower C - Flat 901	+91 98285 55667 rohit.d@mapleheights.org	Owner Resident	0 Members	ACTIVE	

OVERVIEW

TICKETS

FINANCES

BROADCAST

MANAGEMENT

TOWER & WING MANAGEMENT

Society Towers & Unit Allocations

Configure society towers, specify resident capacity, monitor vacancy rates, or add/delete wings.

ADD TOWER

A

Tower A

14 Floors

Residential Tower A (14 Floors, 6 units/floor)

84 Total Units

2 Occupied (2%)

82 Vacant Flats (98%)

Edit

Delete

B

Tower B

14 Floors

Residential Tower B (14 Floors, 6 units/floor)

84 Total Units

2 Occupied (2%)

82 Vacant Flats (98%)

Edit

Delete

C

Tower C

16 Floors

Residential Tower C (16 Floors, 5 units/floor)

80 Total Units

2 Occupied (2%)

78 Vacant Flats (98%)

Edit

Delete

Total Registered Towers: 3

Done

GET/api/flatsList Flats

GET/api/flats/{flat_number}Get Flat

Payments & Maintenance

Maintenance bill generation, dues inspection, simulated payment processing, and ledger CSV exports.

GET/api/payments/myGet My Payments

GET/api/payments/{payment_id}Get Payment Details

POST/api/payments/{payment_id}/payExecute Simulated Payment

GET/api/paymentsList All Payments

POST/api/payments/billsGenerate Society Bills

GET/api/admin/payments/exportExport Payments Csv

Complaints & Tickets

Helpdesk ticketing system for electrical, plumbing, lift, and facility maintenance requests.

POST/api/complaintsCreate Complaint

GET/api/complaints/myGet My Complaints

GET/api/complaints/{ticket_id}Get Complaint By Id

GET/api/admin/complaintsList Admin Complaints

The Society / Apartment Management Portal API

Maple Heights Society Management Portal - Backend API

Authentication: Role-based access control (Resident & Admin)

Resident & Flat Management: Directory, occupancy status, parking allocations

Maintenance & Payments: Monthly billing, simulated payment processing, ledger CSV export

Helpdesk & Complaints: Facility maintenance requests with priority & resolution tracking

Visitor Gate-Passes: Backend-generated QR security passcodes

Amenity Booking: Facility reservations with slot conflict prevention

Broadcast Notices: Digital society noticeboard with urgent alert broadcasting

Admin Operations: Aggregated society analytics and statistics

Snipping Tool

Screenshot copied to clipboard

Automatically saved to screenshots folder.

Mark-up and share

The Society / Apartment Management Portal API2.4.0OAS 3.1

/openapi.json

Maple Heights Society Management Portal - Backend API

Official REST API for the Society / Apartment Management Portal. Built using FastAPI, MongoDB (Motor Async), and JWT Authentication.

Core Modules:

- **Authentication:** Role-based access control (Resident & Admin)
- **Resident & Flat Management:** Directory, occupancy status, parking allocations
- **Maintenance & Payments:** Monthly billing, simulated payment processing, ledger CSV export
- **Helpdesk & Complaints:** Facility maintenance requests with priority & resolution tracking
- **Visitor Gate-Passes:** Backend-generated QR security passcodes
- **Amenity Booking:** Facility reservations with slot conflict prevention
- **Broadcast Notices:** Digital society noticeboard with urgent alert broadcasting
- **Admin Operations:** Aggregated society analytics and statistics

Authorize

Authentication

User authentication, JWT login tokens for residents and administrators, and active session profile lookup.

POST/api/auth/loginLogin

GET/api/auth/meGet My Profile

Complaints & Tickets Helpdesk ticketing system for electrical, plumbing, lift, and facility maintenance requests.

POST	/api/complaints	Create Complaint	
GET	/api/complaints/my	Get My Complaints	
GET	/api/complaints/{ticket_id}	Get Complaint By Id	
GET	/api/admin/complaints	List Admin Complaints	
PATCH	/api/admin/complaints/{ticket_id}/status	Update Complaint Status	

Visitor & Gate Passes Authoritative gate-pass generation with unique security passcodes, QR verification, and visitor status tracking.

POST	/api/gate-passes	Create Gate Pass	
GET	/api/gate-passes/my	Get My Gate Passes	
GET	/api/gate-passes/{id}	Get Gate Pass Details	
GET	/api/admin/gate-passes	List Admin Gate Passes	
PATCH	/api/admin/gate-passes/{id}/status	Update Gate Pass Status	

Amenities Catalog of society facilities (Clubhouse Banquet, Tennis Court, Swimming Pool, BBQ Gazebo, Conference Room).